Community Workshop Manager

Spring Boot, Spring Data JPA and JSP coursework report

1. Overview

Community Workshop Manager is a small web application for a community training centre. Staff use it to keep a list of instructors and the workshops they run. Both records can be created, listed and edited through JSP pages. A third page, the schedule overview, shows the result of a JPQL inner join between the two tables so that each workshop appears next to the details of the person teaching it.

The project uses Java 21, Spring Boot 4.1.2, Spring MVC, Spring Data JPA (Hibernate 7.4), Bean Validation, JSP with JSTL on embedded Tomcat 11.0.27, and an H2 database. It is built with the Maven Wrapper and packaged as an executable WAR file. Deleting records, logins and payments were left out because they were not part of the brief.

2. Entities and relationship

There are exactly two entities. `Instructor` has a generated ID, full name, unique email address and speciality. `Workshop` has a generated ID, unique code, title, scheduled date, duration in minutes, participant capacity and the instructor who teaches it. One instructor can teach many workshops, and each workshop must belong to one existing instructor.

Entity	Attribute	Column and database rule	Form validation
Instructor	id	BIGINT identity, primary key	Not on the form
	fullName	full_name VARCHAR(100) NOT NULL	Required, at most 100 characters
	email	email VARCHAR(120) NOT NULL, unique (uk_instructor_email)	Required, email format, at most 120, stored in lower case
	speciality	speciality VARCHAR(80) NOT NULL	Required, at most 80 characters
Workshop	id	BIGINT identity, primary key	Not on the form
	code	code VARCHAR(20) NOT NULL, unique (uk_workshop_code)	Required, letters, digits and hyphens, stored in upper case
	title	title VARCHAR(120) NOT NULL	Required, at most 120 characters
	scheduledDate	scheduled_date DATE NOT NULL	Required, must parse as a date
	durationMinutes	INT NOT NULL, CHECK BETWEEN 15 AND 480	Required, 15 to 480
	capacity	INT NOT NULL, CHECK BETWEEN 1 AND 200	Required, 1 to 200
	instructor	instructor_id BIGINT NOT NULL, foreign key fk_workshop_instructor	Required, must exist in the database

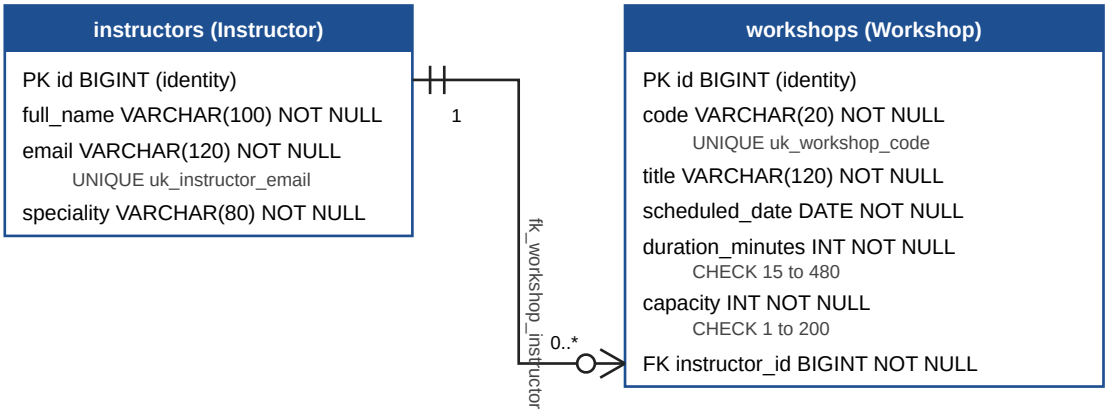


Figure 1. Entity relationship diagram. One instructor teaches zero or more workshops; each workshop has exactly one instructor.

The workshop side owns the relationship, because the foreign key column is in the `workshops` table. It uses `@ManyToOne(optional = false)` with `@JoinColumn(nullable = false)`, so both Hibernate and the database refuse a workshop without an instructor. The instructor side is the inverse `@OneToMany(mappedBy = "instructor")` collection. Constraint names are written out in the annotations so that error handling can recognise them later.

Listing 1. Table, unique and check constraints on Workshop
`src/main/java/ca/communitycentre/workshops/entity/Workshop.java (lines 19 to 23)`

```
@Entity
@Table(name = "workshops",
        uniqueConstraints = @UniqueConstraint(name = "uk_workshop_code", columnNames = "code"))
@Check(name = "ck_workshop_duration", constraints = "duration_minutes BETWEEN 15 AND 480")
@Check(name = "ck_workshop_capacity", constraints = "capacity BETWEEN 1 AND 200")
```

Listing 2. Owning many-to-one side with a non-null foreign key
`src/main/java/ca/communitycentre/workshops/entity/Workshop.java (lines 50 to 54)`

```
// Owning side: every workshop must point at an existing instructor row.
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "instructor_id", nullable = false,
        foreignKey = @ForeignKey(name = "fk_workshop_instructor"))
private Instructor instructor;
```

Listing 3. Inverse one-to-many side on Instructor
`src/main/java/ca/communitycentre/workshops/entity/Instructor.java (lines 34 to 37)`

```
// Inverse side of the relationship. The foreign key lives in the workshops table.
@OneToMany(mappedBy = "instructor")
@OrderBy("scheduledDate ASC")
private List<Workshop> workshops = new ArrayList<>();
```

Hibernate creates the tables from these classes (`ddl-auto=update`). Querying H2's information schema after a fresh start confirmed that the constraints exist in the database itself:

```
TABLE_NAME  | CONSTRAINT_NAME        | CONSTRAINT_TYPE
INSTRUCTORS | CONSTRAINT_9           | PRIMARY KEY
INSTRUCTORS | UK_INSTRUCTOR_EMAIL    | UNIQUE
WORKSHOPS   | CK_WORKSHOP_DURATION   | CHECK
WORKSHOPS   | CK_WORKSHOP_CAPACITY   | CHECK
WORKSHOPS   | FK_WORKSHOP_INSTRUCTOR | FOREIGN KEY
WORKSHOPS   | CONSTRAINT_E           | PRIMARY KEY
WORKSHOPS   | UK_WORKSHOP_CODE       | UNIQUE
(7 rows, 26 ms)
CONSTRAINT_NAME      | CHECK_CLAUSE
CK_WORKSHOP_DURATION | "DURATION_MINUTES" BETWEEN 15 AND 480
CK_WORKSHOP_CAPACITY | "CAPACITY" BETWEEN 1 AND 200
(2 rows, 1 ms)
```

3. Architecture

The code is split into packages by role: `entity`, `repository`, `projection`, `service`, `controller`, `form`, `exception` and `config`. Every class receives its dependencies through its constructor.

Layer	Classes	Responsibility
Repository	InstructorRepository, WorkshopRepository	Extend `JpaRepository`. Derived queries for sorting and uniqueness checks, an entity graph that loads each workshop's instructor, and the `@Query` inner join.
Service	InstructorService, WorkshopService	Business rules: normalising text, checking unique values, resolving the chosen instructor, and running writes inside `@Transactional` methods. Read methods are `readOnly`.
Controller	InstructorController, WorkshopController, OverviewController, GlobalExceptionHandler	Bind and validate form models, call the services, add results to the model, and choose the JSP view or redirect.
View	JSP pages in `WEB-INF/jsp`	Lists, forms, the overview table and error pages, using JSTL, EL and Spring form tags with a shared header and stylesheet.

A request for the workshop list goes from `WorkshopController.list()` to `WorkshopService.findAll()`, then to `WorkshopRepository.findAllByOrderByScheduledDateAscCodeAsc()`, and the result is placed in the model as `workshops` for `workshops/list.jsp`. Open Session in View is switched off, so the repository method uses `@EntityGraph(attributePaths = "instructor")` to fetch the instructor in the same SQL statement instead of relying on lazy loading while the JSP renders.

JSP needs WAR packaging. The application class extends `SpringBootServletInitializer`, and the Tomcat runtime and Jasper are declared with `provided` scope, so the repackaged WAR keeps them in `WEB-INF/lib-provided` and still starts with `java -jar`.

Listing 4. Embedded Tomcat, Jasper and JSTL dependencies
pom.xml (lines 39 to 60)

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-tomcat-runtime</artifactId>
    <scope>provided</scope>
</dependency>
<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
    <scope>provided</scope>
</dependency>

<dependency>
    <groupId>jakarta.servlet.jsp.jstl</groupId>
    <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
</dependency>
<dependency>
    <groupId>org.glassfish.web</groupId>
    <artifactId>jakarta.servlet.jsp.jstl</artifactId>
</dependency>
```

4. Sample data

`DataSeeder` is an `ApplicationRunner` that only inserts data when both tables are empty, so restarting the application never creates duplicates. It adds ten instructors and ten workshops with fictional names. Amara Okafor and Daniel Reyes teach two workshops each, six instructors teach one, and Lucas Moreau and Sofia Petrova have none.

Listing 5. Guard that prevents duplicate seeding
src/main/java/ca/communitycentre/workshops/config/DataSeeder.java (lines 44 to 47)

```
public boolean seed() {
    if (instructorRepository.count() > 0 || workshopRepository.count() > 0) {
        log.info("Sample data skipped: database already contains records.");
        return false;
```

The application was started against an empty database folder, stopped, and then queried with the H2 shell. The log printed `Sample data loaded: 10 instructors and 10 workshops.` and the query returned:

```
INSTRUCTORS | WORKSHOPS
10          | 10
(1 row, 15 ms)
FULL_NAME        | WORKSHOPS
Amara Okafor     | 2
Daniel Reyes     | 2
Grace Mensah     | 1
Hannah Lindqvist | 1
Mei Chen         | 1
Omar Haddad      | 1
Priya Nair       | 1
Tomasz Kowalski  | 1
Lucas Moreau     | 0
Sofia Petrova    | 0
(10 rows, 10 ms)
```

Starting the application again on the same folder logged `Sample data skipped: database already contains records.` and the counts did not change.

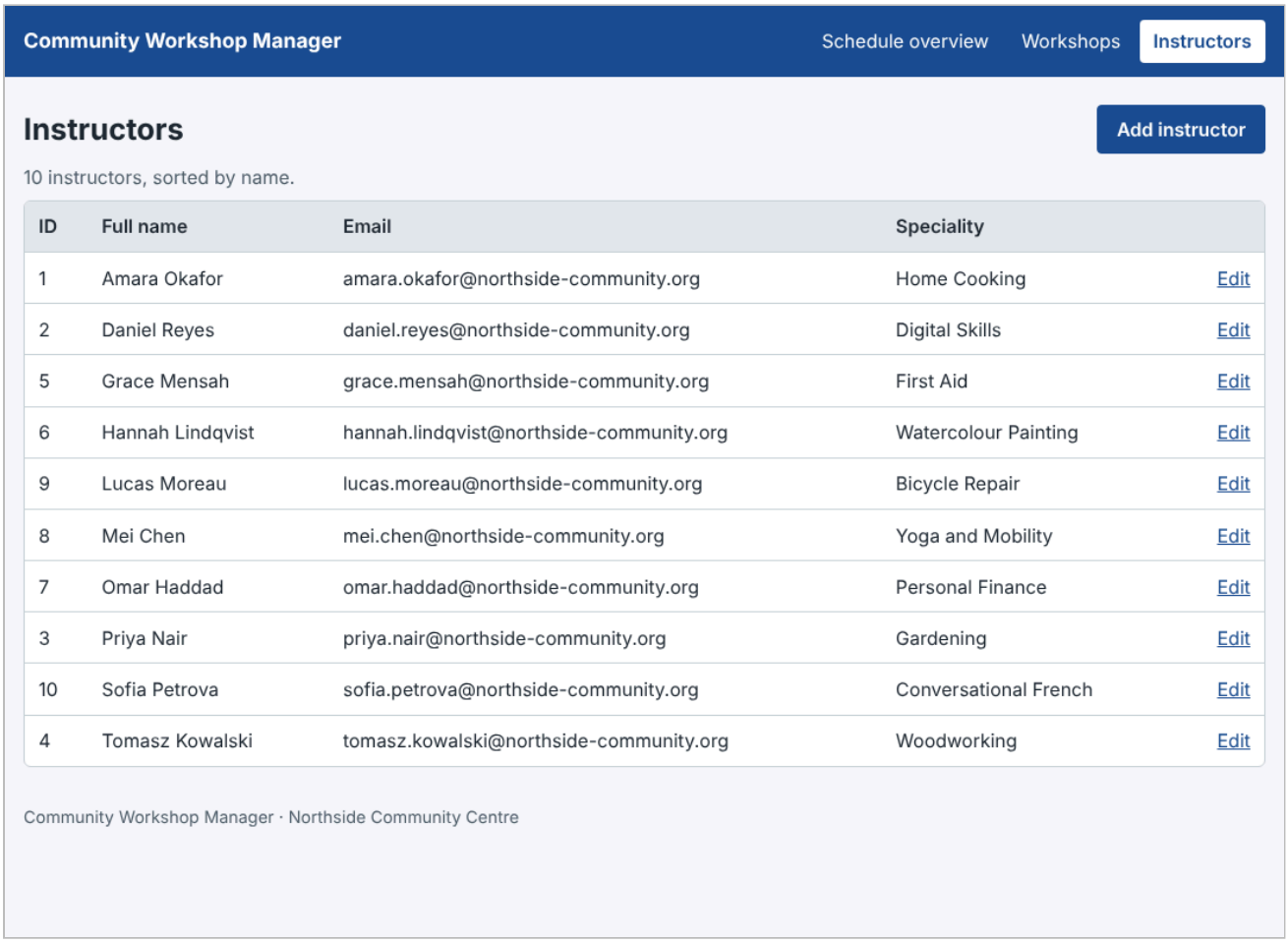


Figure 2. Instructor list on a fresh database, showing the ten seeded instructors sorted by name.

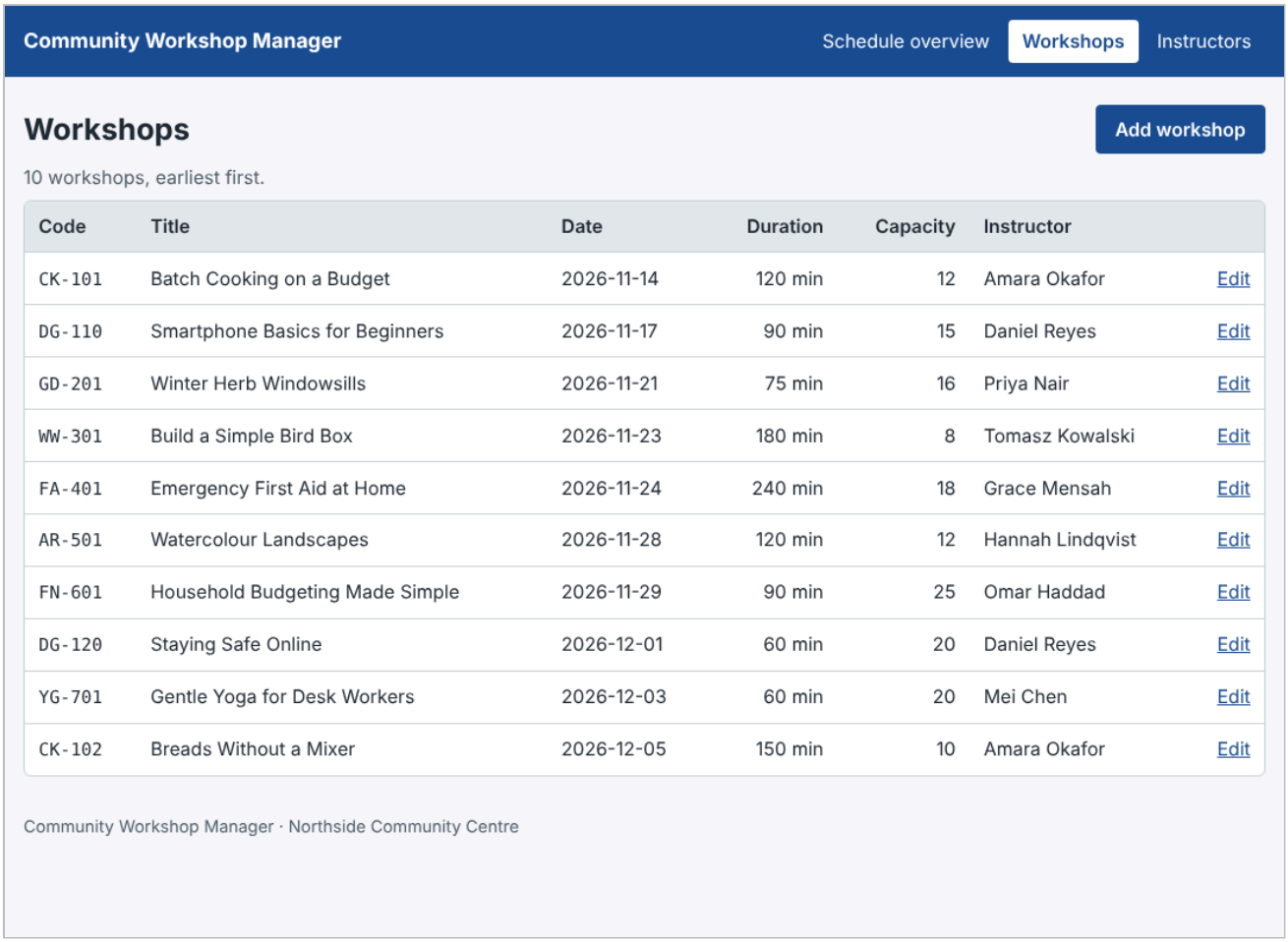


Figure 3. Workshop list on a fresh database, showing the ten seeded workshops with their instructors.

5. Create

Each create form posts to a controller method that validates a dedicated form model (`InstructorForm` or `WorkshopForm`). The form classes contain only the fields a user may edit, so a request parameter such as `id` has nothing to bind to and cannot change a generated key. If validation passes, the controller calls the service and then redirects to the list with a flash message (Post/Redirect/Get), so refreshing the page does not submit the form again.

Listing 6. Instructor create handler with validation, error handling and redirect
src/main/java/ca/communitycentre/workshops/controller/InstructorController.java (lines 45 to 63)

```
@PostMapping
public String create(@Valid @ModelAttribute("instructorForm") InstructorForm form, BindingResult
result,
                     RedirectAttributes redirectAttributes) {
    if (result.hasErrors()) {
        return FORM_VIEW;
    }
    try {
        Instructor saved = instructorService.create(form);
        redirectAttributes.addFlashAttribute("successMessage",
                "Instructor " + saved.getFullName() + " was added.");
        return "redirect:/instructors";
    } catch (FieldValidationException ex) {
        FormErrors.apply(result, ex);
    } catch (DataIntegrityViolationException ex) {
        // Raised on flush or commit if the database rejects the row.
        FormErrors.apply(result, IntegrityViolationTranslator.translate(ex));
    }
    return FORM_VIEW;
}
```

The service normalises the text before it checks for duplicates, so `Ruth@Example.org` and `ruth@example.org` are treated as the same address. `saveAndFlush` is used so that a database constraint failure happens inside the service call rather than at some later point.

Listing 7. InstructorService.create
src/main/java/ca/communitycentre/workshops/service/InstructorService.java (lines 36 to 45)

```
public Instructor create(InstructorForm form) {
    String email = TextNormalizer.email(form.getEmail());
    if (instructorRepository.existsByEmail(email)) {
        throw new DuplicateValueException("email", DUPLICATE_EMAIL);
    }
    Instructor instructor = new Instructor(
            TextNormalizer.clean(form.getFullName()), email,
TextNormalizer.clean(form.getSpeciality()));
    // Flush now so a constraint failure is raised inside this call, not later.
    return instructorRepository.saveAndFlush(instructor);
}
```

Creating a workshop works the same way, with one extra step: the service looks up the chosen instructor ID and throws `InvalidInstructorException` if it does not exist. The workshop form fills its drop-down from `instructorService.findAll()` every time it is shown, including after a failed submission.

Listing 8. WorkshopService.create resolving the instructor from the database
src/main/java/ca/communitycentre/workshops/service/WorkshopService.java (lines 47 to 56)

```
public Workshop create(WorkshopForm form) {
    String code = TextNormalizer.code(form.getCode());
    if (workshopRepository.existsByCode(code)) {
        throw new DuplicateValueException("code", DUPLICATE_CODE);
    }
    Instructor instructor = requireInstructor(form.getInstructorId());
    Workshop workshop = new Workshop(code, TextNormalizer.clean(form.getTitle()),
form.getScheduledDate(),
            form.getDurationMinutes(), form.getCapacity(), instructor);
    return workshopRepository.saveAndFlush(workshop);
}
```

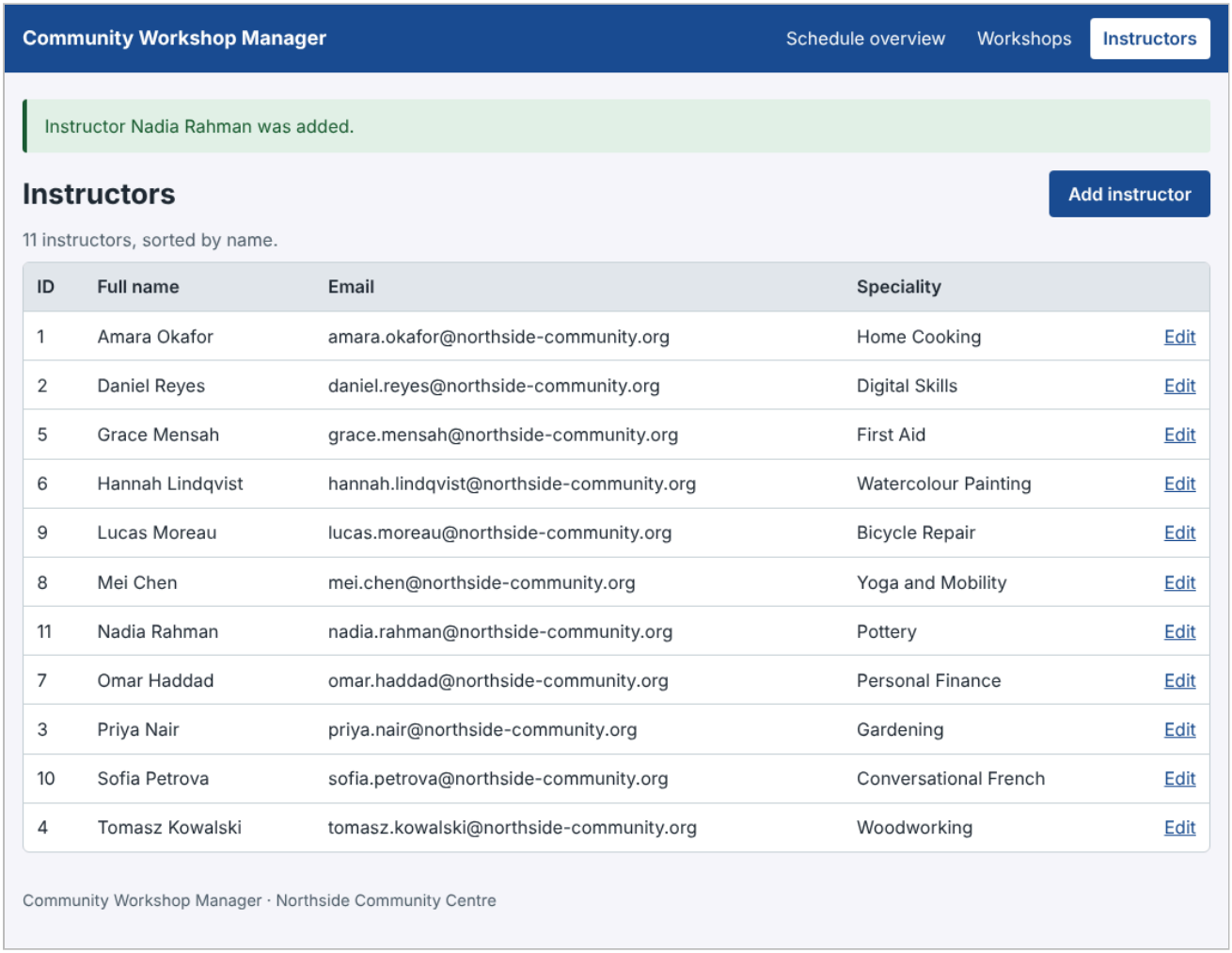


Figure 4. After creating Nadia Rahman. Extra spaces in the name were removed and the email was saved in lower case.

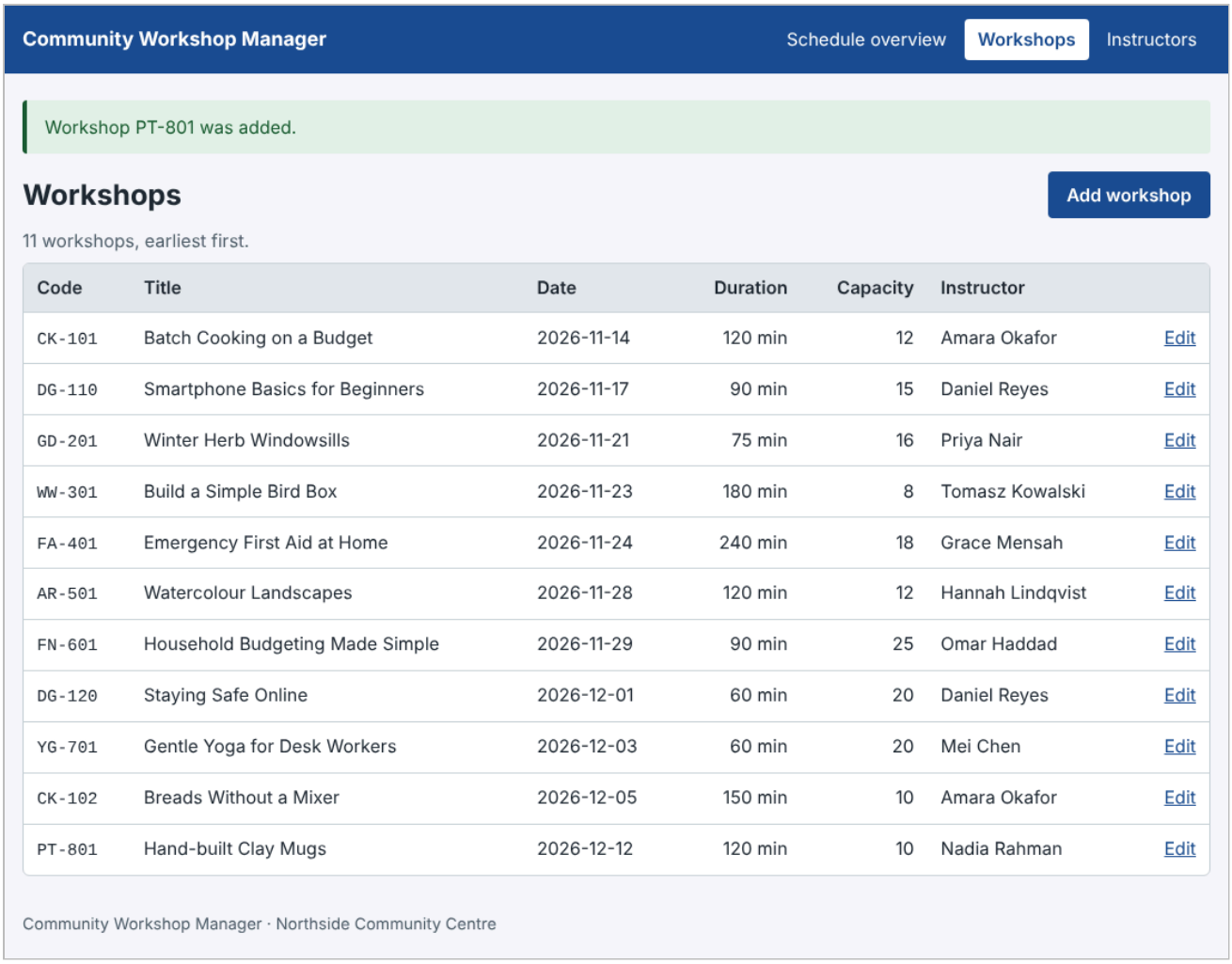


Figure 5. Workshop PT-801 added for Nadia Rahman. The code was typed as "pt-801" and stored in capitals.

6. Read and the inner join

The list pages read all records through the services and show them in tables with an edit link per row. Empty lists show a short message with a link to the create form. All user-entered values are printed with `<c:out>`, which escapes HTML; a test name containing a `<script>` tag was displayed as plain text.

The overview uses an explicit JPQL inner join written in a repository method. It starts from `Instructor` and joins its `workshops` collection, and a constructor expression builds a `WorkshopOverview` record for each row. Because it is an inner join, an instructor with no workshops produces no rows.

Listing 9. Inner join query with a typed projection
src/main/java/ca/communitycentre/workshops/repository/InstructorRepository.java (lines 23 to 31)

```
@Query("""
        SELECT new ca.communitycentre.workshops.projection.WorkshopOverview(
            i.id, i.fullName, i.email, i.speciality,
            w.id, w.code, w.title, w.scheduledDate, w.durationMinutes, w.capacity)
        FROM Instructor i
        INNER JOIN i.workshops w
        ORDER BY w.scheduledDate ASC, w.code ASC
        """)
List<WorkshopOverview> findWorkshopOverview();
```

Listing 10. Projection record returned by the join
src/main/java/ca/communitycentre/workshops/projection/WorkshopOverview.java (lines 9 to 20)

```
public record WorkshopOverview(
        Long instructorId,
        String instructorName,
        String instructorEmail,
        String speciality,
        Long workshopId,
        String workshopCode,
        String workshopTitle,
        LocalDate scheduledDate,
        int durationMinutes,
        int capacity) {
}
```

The rows pass through `WorkshopService.getOverview()` and `OverviewController` into `overview.jsp`, which loops over them with `<c:forEach>`. Record components are read with method calls in EL, for example `${row.instructorName()}`.

Listing 11. Controller method for the overview page
src/main/java/ca/communitycentre/workshops/controller/OverviewController.java (lines 23 to 27)

```
@GetMapping("/overview")
public String overview(Model model) {
    model.addAttribute("rows", workshopService.getOverview());
    return "overview";
}
```

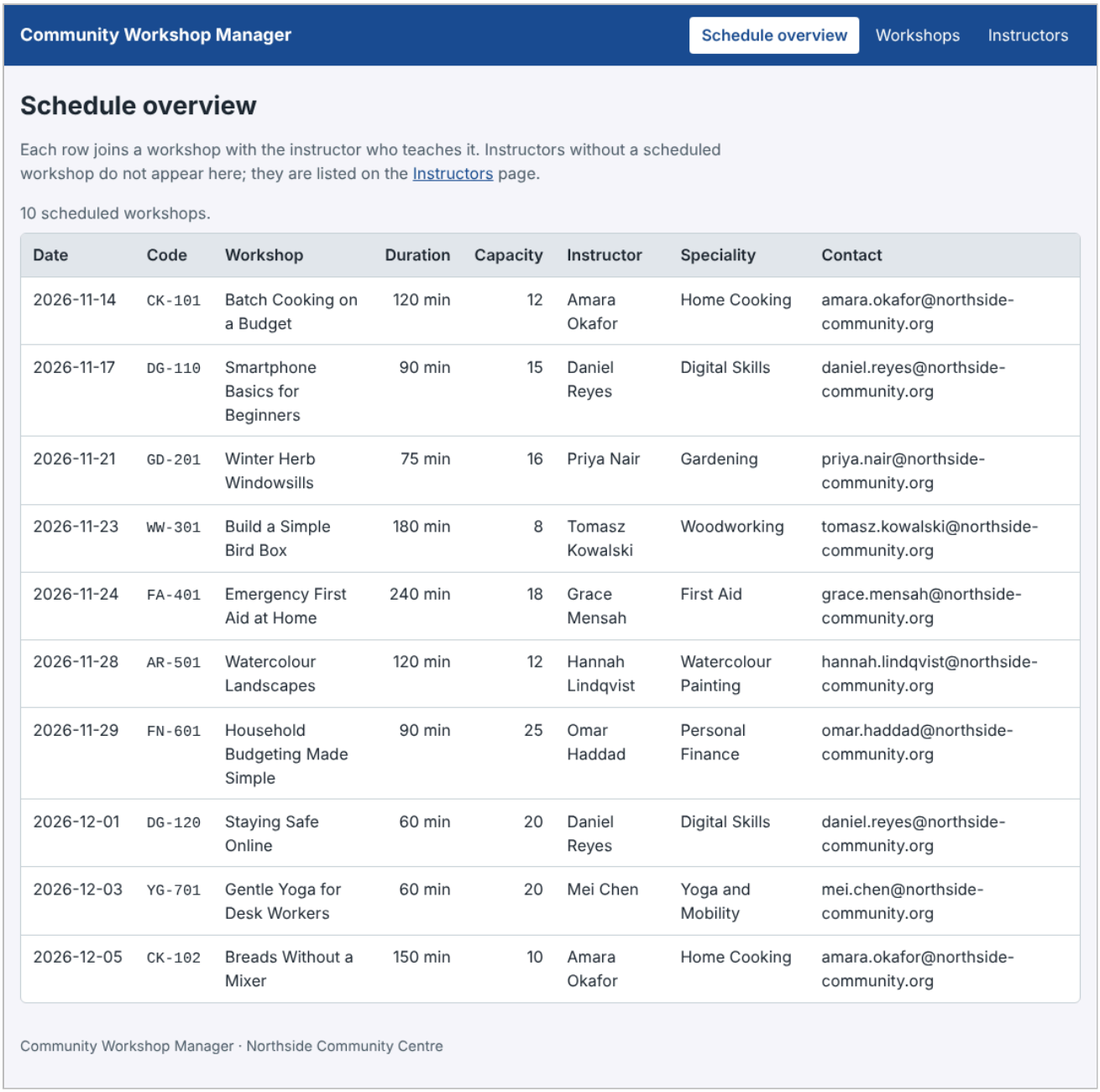


Figure 6. Schedule overview on a fresh database: ten joined rows. Lucas Moreau and Sofia Petrova have no workshops and are not listed.

7. Update

The edit form is filled from the stored entity through `InstructorForm.from()` or `WorkshopForm.from()`. The record ID comes from the URL path, not from the form. On submit, the service loads the existing managed entity, copies the new values onto it and flushes. No new object is created, so the same row is updated. The uniqueness check excludes the record being edited, so saving an instructor with their own email address, or a workshop with its own code, is allowed.

Listing 12. WorkshopService.update, including changing the instructor
src/main/java/ca/communitycentre/workshops/service/WorkshopService.java (lines 59 to 74)

```
public Workshop update(Long id, WorkshopForm form) {
    Workshop workshop = findById(id);
    String code = TextNormalizer.code(form.getCode());
    if (workshopRepository.existsByCodeAndIdNot(code, id)) {
        throw new DuplicateValueException("code", DUPLICATE_CODE);
    }
    Instructor instructor = requireInstructor(form.getInstructorId());
    workshop.setCode(code);
    workshop.setTitle(TextNormalizer.clean(form.getTitle()));
    workshop.setScheduledDate(form.getScheduledDate());
    workshop.setDurationMinutes(form.getDurationMinutes());
    workshop.setCapacity(form.getCapacity());
    workshop.setInstructor(instructor);
    workshopRepository.flush();
    return workshop;
}
```

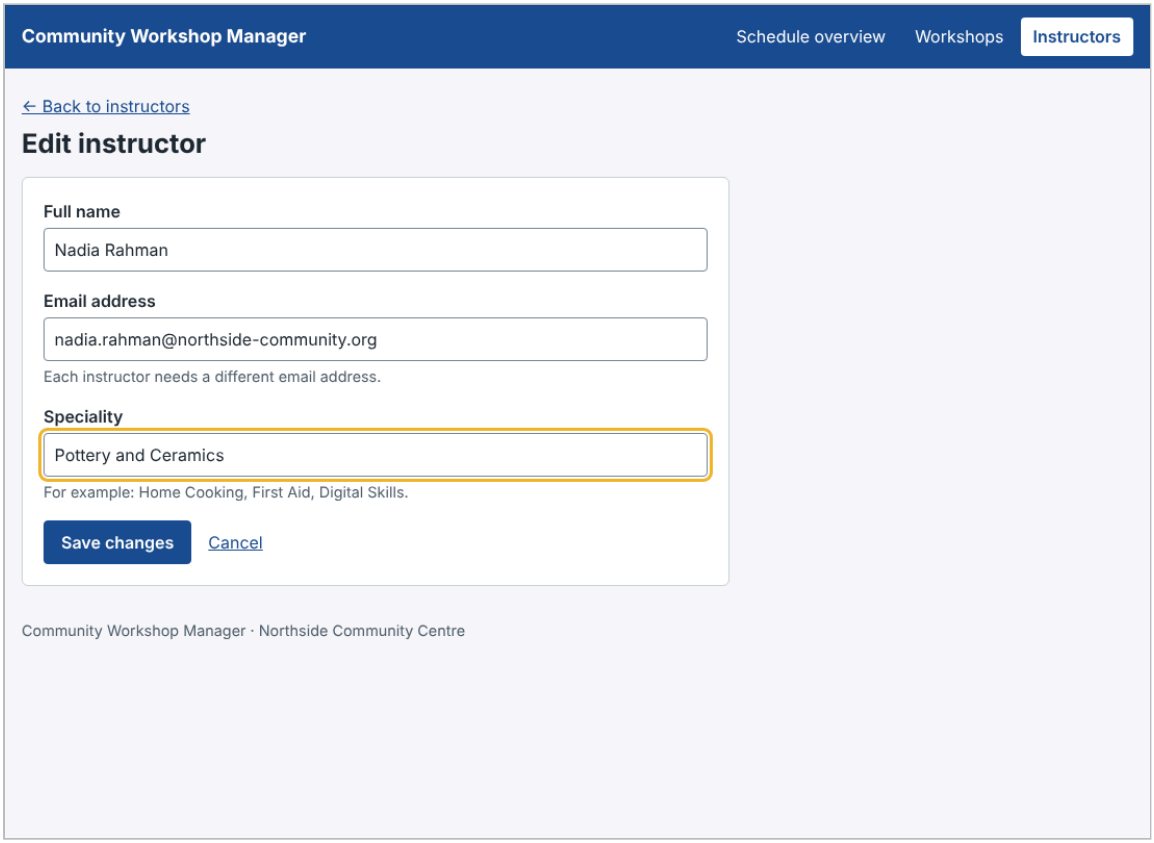


Figure 7. Edit form for Nadia Rahman, filled from the database, with the speciality being changed.

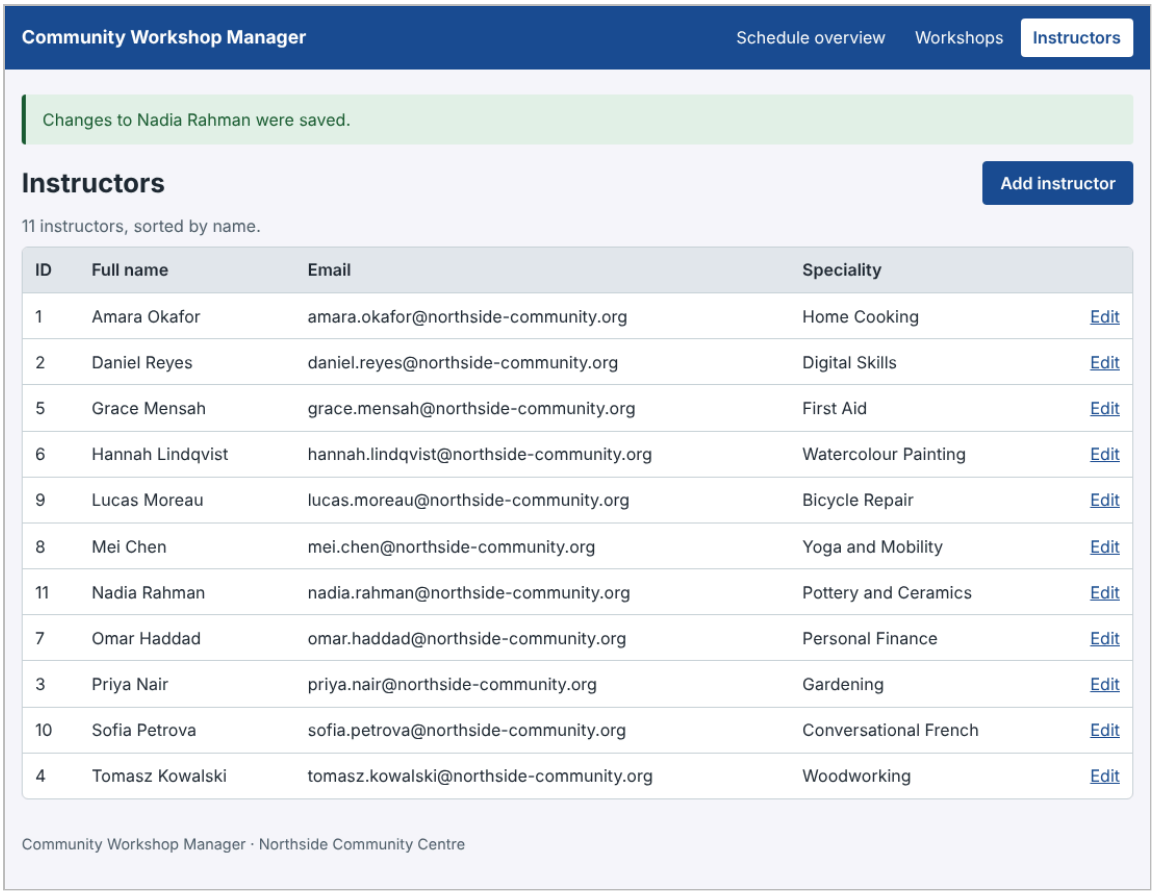


Figure 8. The list after saving. The row count stayed at 11 and the email was accepted unchanged.

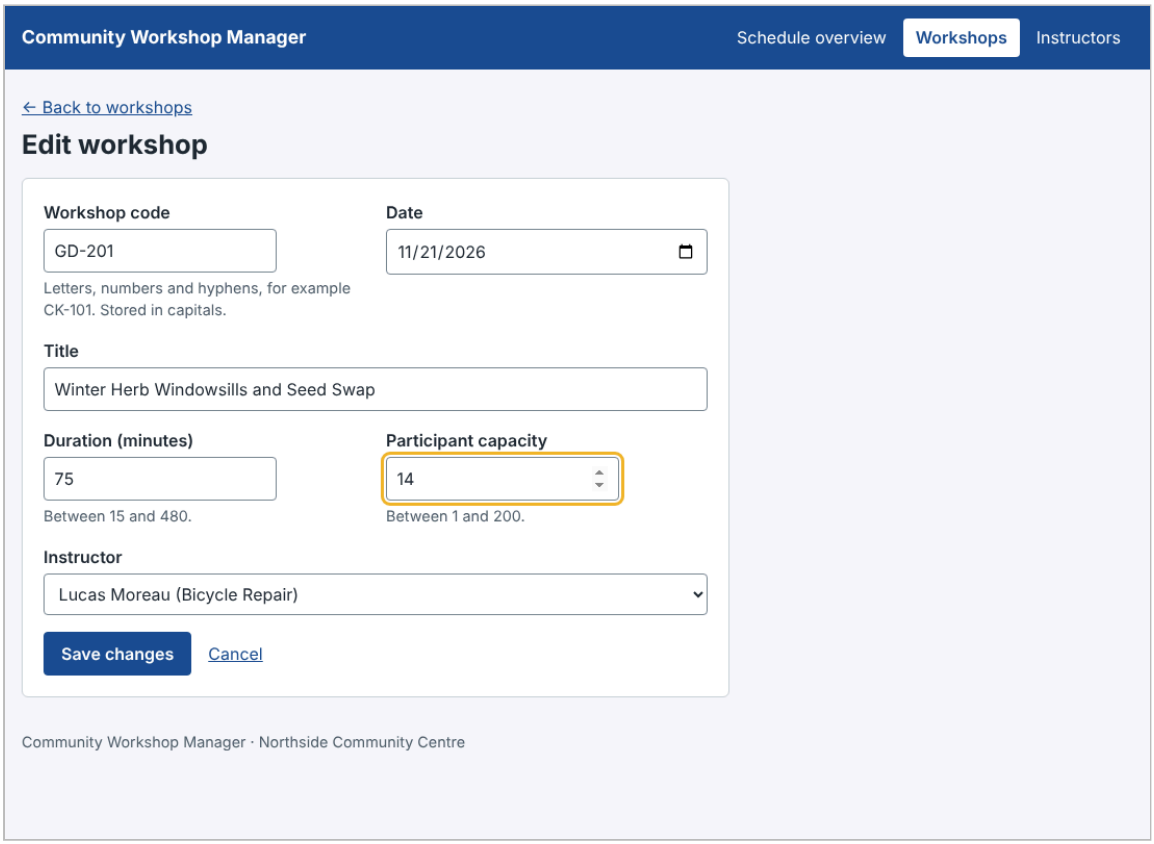


Figure 9. Editing GD-201: new title, capacity and instructor (Priya Nair to Lucas Moreau).

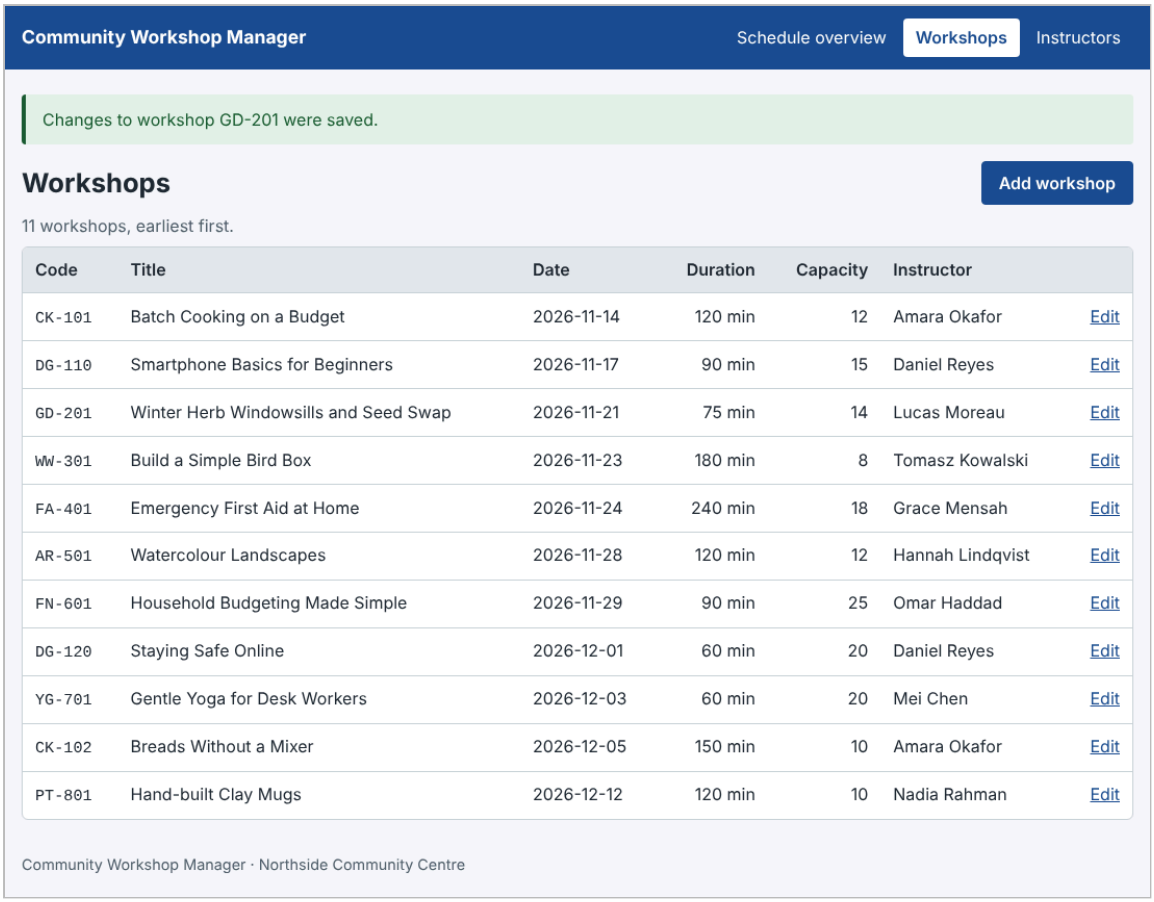


Figure 10. GD-201 after saving, now taught by Lucas Moreau, with its code kept.

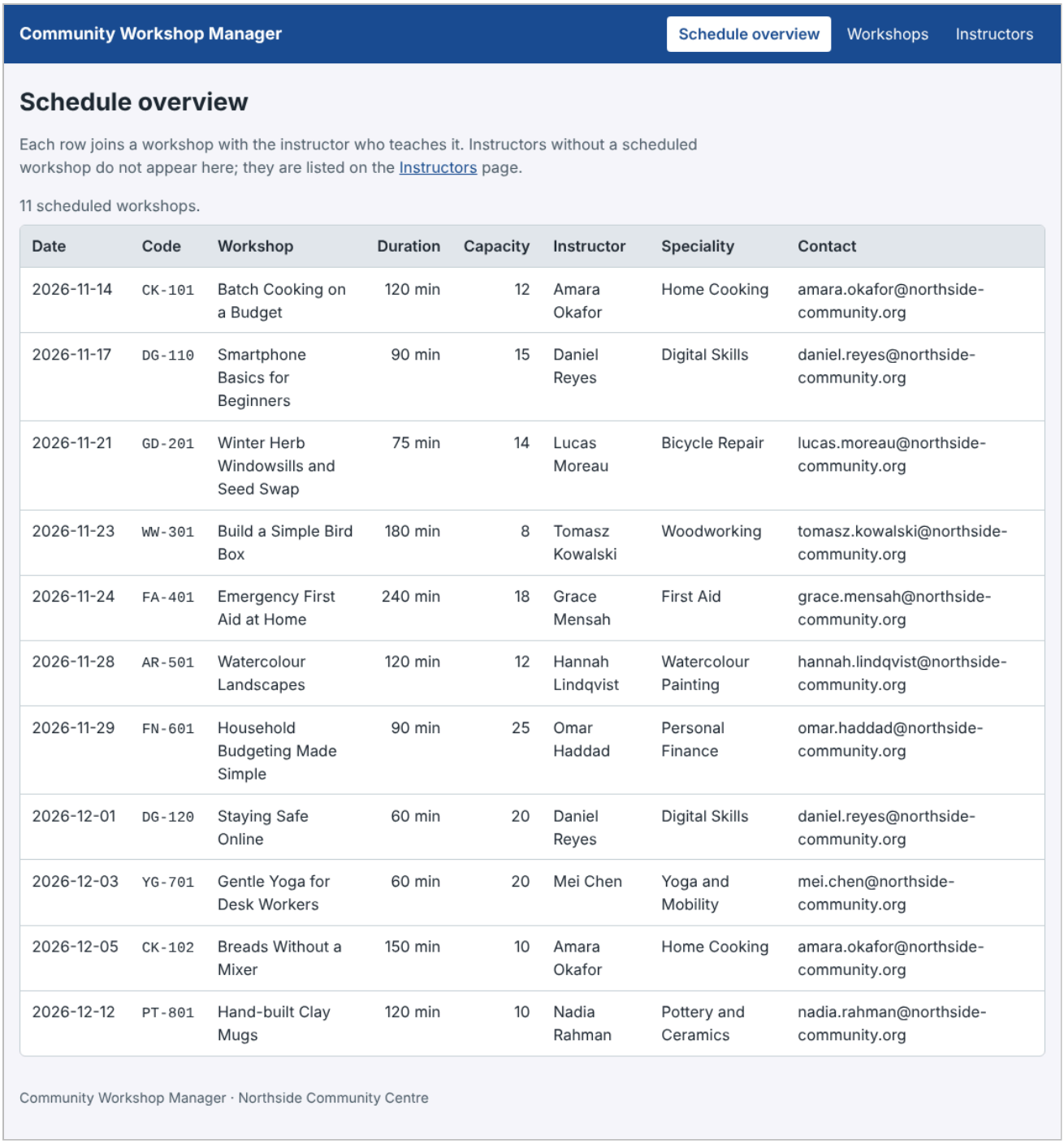


Figure 11. Overview after the edits. Lucas Moreau now appears. Priya Nair no longer teaches anything, so the inner join leaves her out.

8. Validation and integrity errors

Validation happens in three places. Bean Validation annotations on the form models check required fields, lengths, email format, the code pattern and the numeric ranges. The services check unique values and whether the instructor exists, and report problems as a `FieldValidationException` that names the form field. Finally, the database enforces the same rules with NOT NULL, unique, check and foreign key constraints. Date and number conversion errors use messages from `messages.properties`, such as "Enter a valid date."

Listing 13. Range and required checks on WorkshopForm
src/main/java/ca/communitycentre/workshops/form/WorkshopForm.java (lines 35 to 46)

```
@NotNull(message = "Enter the duration in minutes.")
@Min(value = Workshop.MIN_DURATION, message = "Duration must be at least 15 minutes.")
@Max(value = Workshop.MAX_DURATION, message = "Duration cannot be more than 480 minutes.")
private Integer durationMinutes;

@NotNull(message = "Enter the participant capacity.")
@Min(value = Workshop.MIN_CAPACITY, message = "Capacity must be at least 1 participant.")
@Max(value = Workshop.MAX_CAPACITY, message = "Capacity cannot be more than 200 participants.")
private Integer capacity;

@NotNull(message = "Choose an instructor.")
private Long instructorId;
```

The service check for a duplicate email cannot fully prevent a race, because two requests could pass the check at the same time. In that case the unique constraint fails during flush or commit and Spring throws `DataIntegrityViolationException`. The controller catches it. Since the controller is outside the `@Transactional` service method, the failed transaction has already been rolled back and nothing else is written in it. `IntegrityViolationTranslator` looks for the known constraint names and turns them into a field message. Any other constraint gives a general message. SQL text and stack traces are never shown; `server.error.include-stacktrace=never` is also set.

Listing 14. Mapping constraint names to form messages
src/main/java/ca/communitycentre/workshops/exception/IntegrityViolationTranslator.java (lines 25 to 37)

```
public static FieldValidationException translate(DataIntegrityViolationException ex) {
    String detail = collectMessages(ex).toLowerCase(Locale.ROOT);
    if (detail.contains("uk_instructor_email")) {
        return new DuplicateValueException("email", "Another instructor already uses this email
address.");
    }
    if (detail.contains("uk_workshop_code")) {
        return new DuplicateValueException("code", "Another workshop already uses this code.");
    }
    if (detail.contains("fk_workshop_instructor")) {
        return new FieldValidationException("instructorId", "The selected instructor does not
exist.");
    }
    return new FieldValidationException(null, GENERAL_MESSAGE);
}
```

Unknown record IDs raise `RecordNotFoundException`, which `GlobalExceptionHandler` turns into a 404 page. A non-numeric ID in the URL is handled the same way.

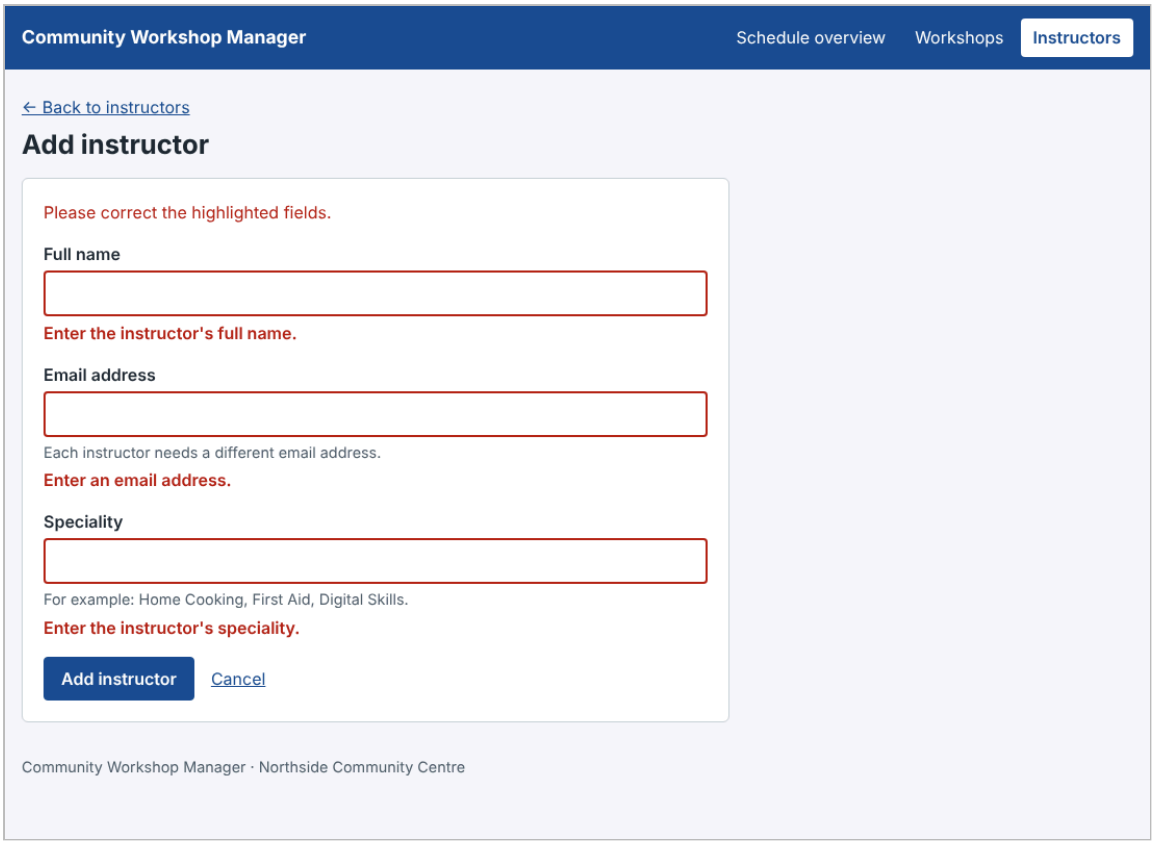


Figure 12. Submitting an empty instructor form shows a message under each required field.

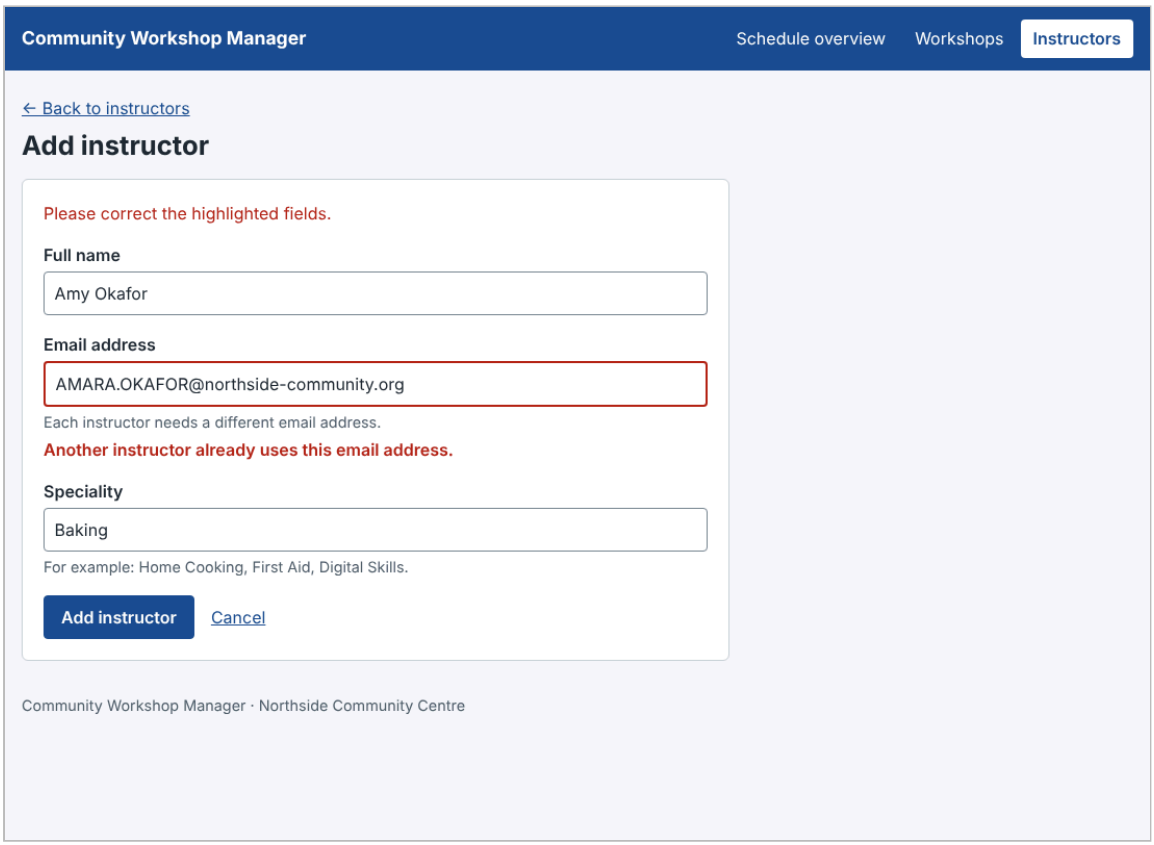


Figure 13. Duplicate email typed in capitals is still rejected, and the other values are kept.

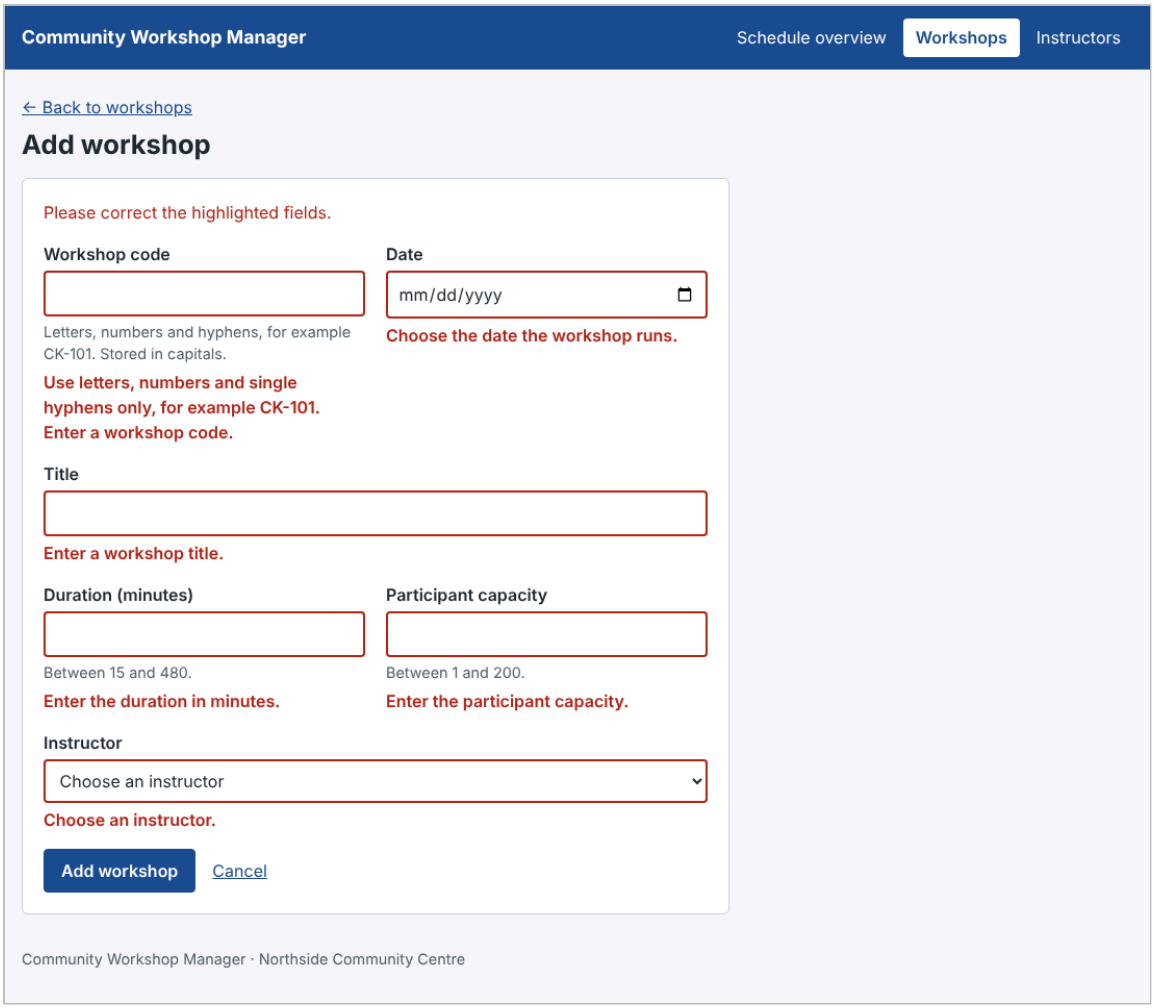


Figure 14. Empty workshop form: all six required fields are reported.

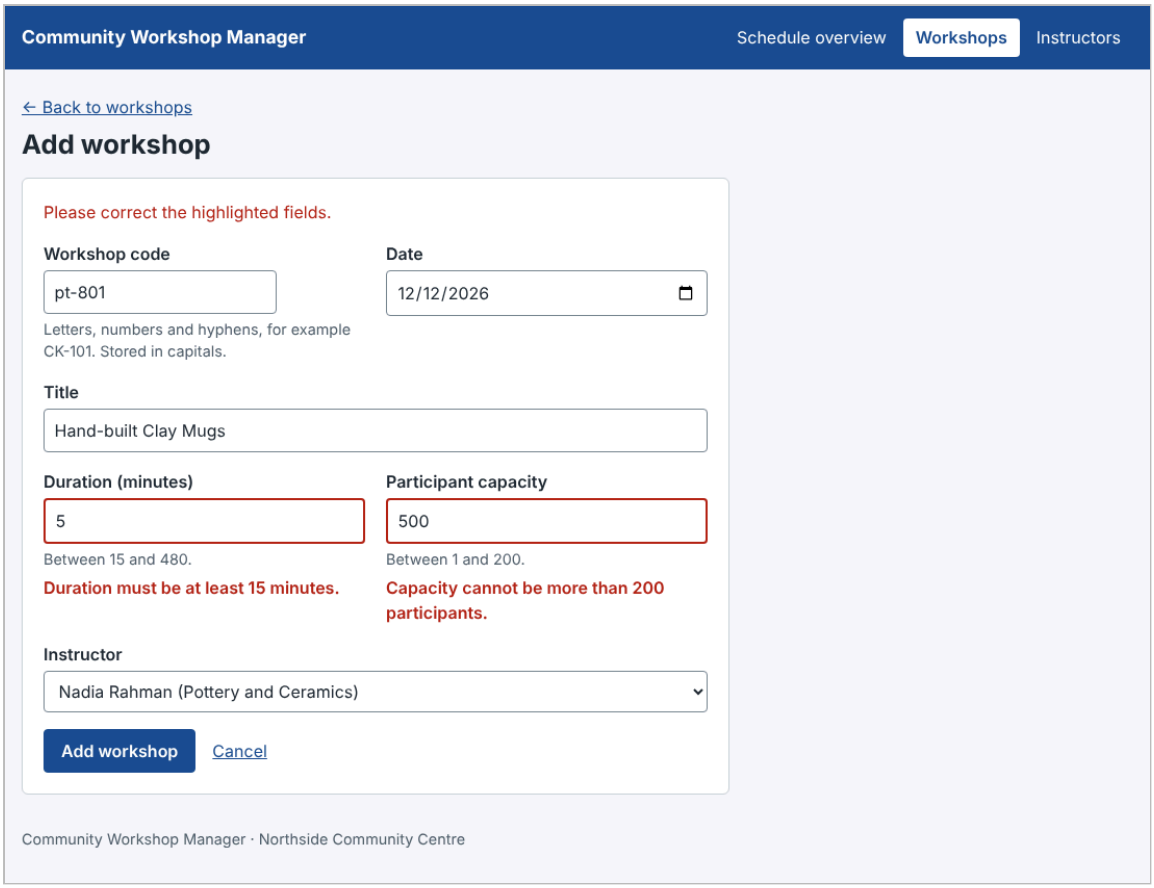


Figure 15. Out-of-range duration and capacity. Entered values and the selected instructor are kept.

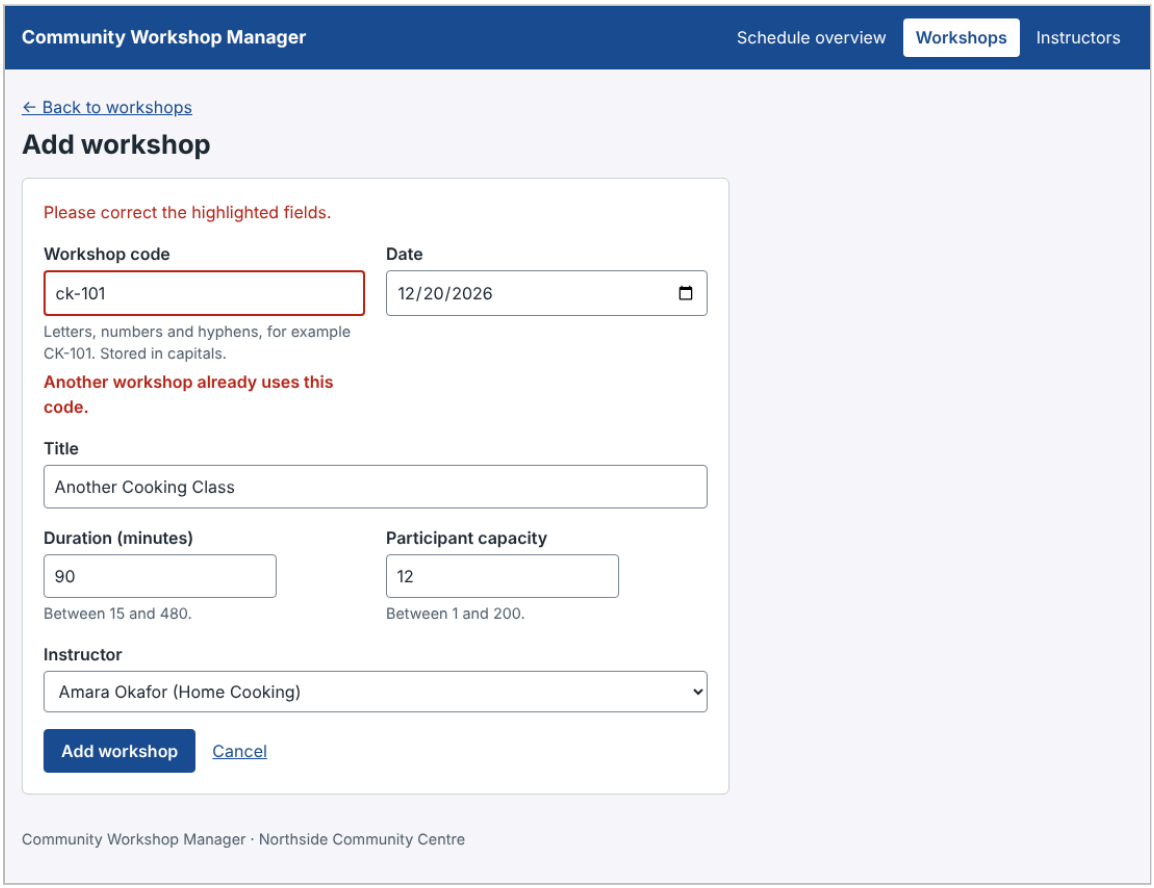


Figure 16. Workshop code "ck-101" is rejected because CK-101 already exists.

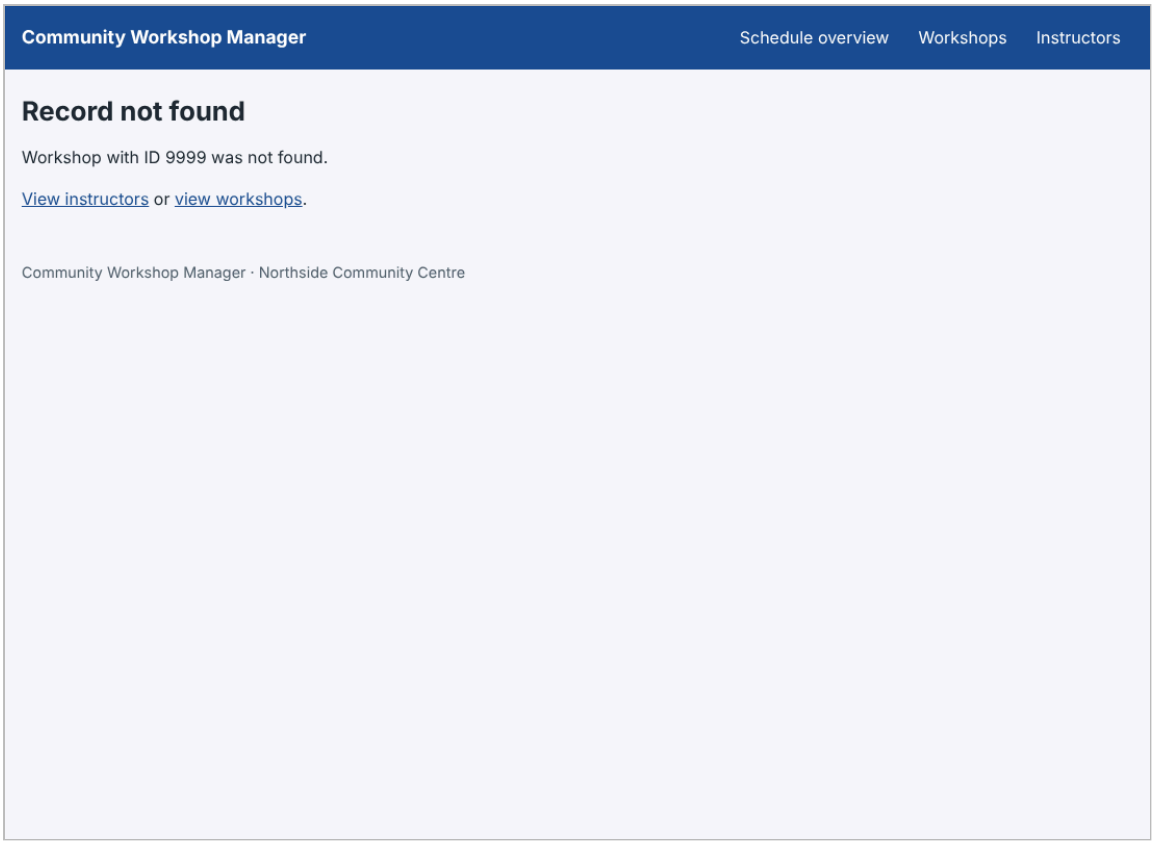


Figure 17. Requesting /workshops/9999/edit shows a 404 page instead of an error trace.

9. Testing

Repository tests use `@DataJpaTest` with a real in-memory H2 database, so the queries and constraints are actually executed. Tests that check stored values call `flush()` and `clear()` on the persistence context first, so the data is read back from the database instead of from Hibernate's cache. Service tests use Mockito mocks of the repositories to check the business rules in isolation; they do not test SQL. Controller tests use `@WebMvcTest` with `@MockitoBean` services. Two `@SpringBootTest` tests run the full stack: one starts embedded Tomcat on a random port and requests real JSP pages, and the other forces the unique constraint to fail on flush to check the error path end to end.

Test class	Tests	What it covers
InstructorRepositoryTest	9	Create and read, update without a new row, unique email, uniqueness check excluding own ID, missing ID, sort order, inverse collection, join fields, join excludes instructors without workshops
WorkshopRepositoryTest	9	Create with stored foreign key, update and change instructor, unique code, own code allowed, null instructor rejected, unknown instructor ID rejected by the foreign key, capacity check constraint, missing ID, entity graph loading
DataSeederTest	1	Ten of each on an empty database, second run skipped, two instructors with no workshops
InstructorServiceTest	6	Normalisation, duplicate email, missing ID, update in place keeping own email, duplicate on update

WorkshopServiceTest	8	Instructor lookup, invalid instructor, duplicate code, update changing instructor, missing workshop, overview delegation
InstructorControllerTest	10	Required fields, email format and length, redirect and flash message, duplicate and commit-time constraint errors, unknown and non-numeric IDs, ignored `id` parameter
WorkshopControllerTest	8	Instructor choices, kept values after errors, date binding errors, ranges, code pattern, invalid instructor, update redirect
JspRenderingIntegrationTest	4	Real Tomcat and Jasper render the lists, overview, form and error pages
IntegrityHandlingIntegrationTest	1	Unique constraint failing on flush becomes a form error and the row is rolled back

Listing 15. Repository test for the inner join
`src/test/java/ca/communitycentre/workshops/repository/InstructorRepositoryTest.java (lines 144 to 161)`

```
void overviewExcludesInstructorsWithoutWorkshopsAndRepeatsThoseWithSeveral() {
    Instructor ruth = entityManager.persist(new Instructor("Ruth Adeyemi", "ruth@example.org",
"Sewing"));
    Instructor idle = entityManager.persist(new Instructor("Ivan Idle", "ivan@example.org",
"Chess"));
    Instructor sam = entityManager.persist(new Instructor("Sam Ortiz", "sam@example.org",
"Pottery"));
    entityManager.persist(new Workshop("SW-2", "Darning Socks", LocalDate.of(2026, 12, 8), 45, 8,
ruth));
    entityManager.persist(new Workshop("SW-1", "Hemming Trousers", LocalDate.of(2026, 12, 1), 60,
8, ruth));
    entityManager.persist(new Workshop("PT-1", "Hand-built Bowls", LocalDate.of(2026, 12, 3), 120,
6, sam));
    entityManager.flush();
    entityManager.clear();

    List<WorkshopOverview> rows = instructorRepository.findWorkshopOverview();

    assertThat(rows).extracting(WorkshopOverview::workshopCode).containsExactly("SW-1", "PT-1",
"SW-2");
    assertThat(rows).extracting(WorkshopOverview::instructorId)
            .containsOnly(ruth.getId(), sam.getId())
            .doesNotContain(idle.getId());
    assertThat(instructorRepository.count()).isEqualTo(3);
}
```

Results. `./mvnw clean test` ran 56 tests with 0 failures, 0 errors and 0 skipped (BUILD SUCCESS). `./mvnw clean package` produced `target/community-workshop-manager.war`.

The packaged WAR was then started with `java -jar` against a new, empty database folder and tested in headless Chromium with a Playwright script. The script performed 18 checks, all of which passed: the seeded lists and overview, creating and editing both entities, changing a workshop's instructor, required and range validation, duplicate email and code (on create and update), kept form values, the 404 page, visible keyboard focus, a 390 pixel

wide viewport without horizontal scrolling, and no browser console errors. After restarting the application on the same database, all changes were still there and the seeder did not run again. The application logs contained no warnings or errors. Every screenshot in this report was taken by that script from the running application; for the desktop views only the empty page margins at the left and right were trimmed.

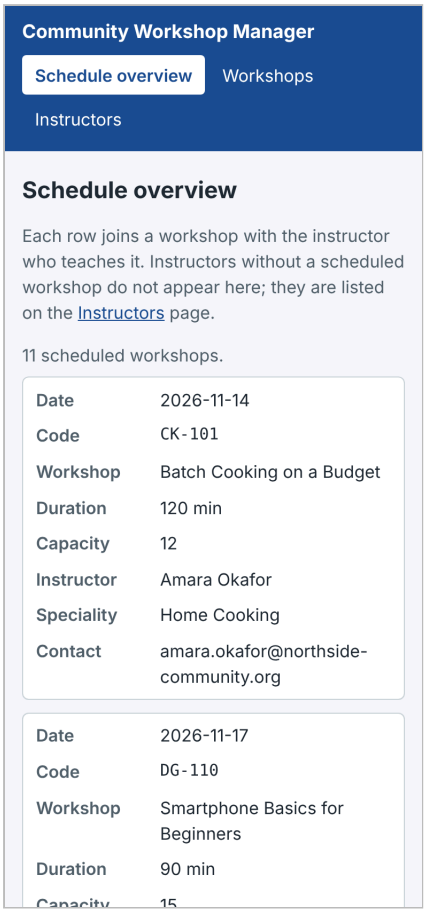


Figure 18. Overview at 390 px width. Table rows become labelled cards.

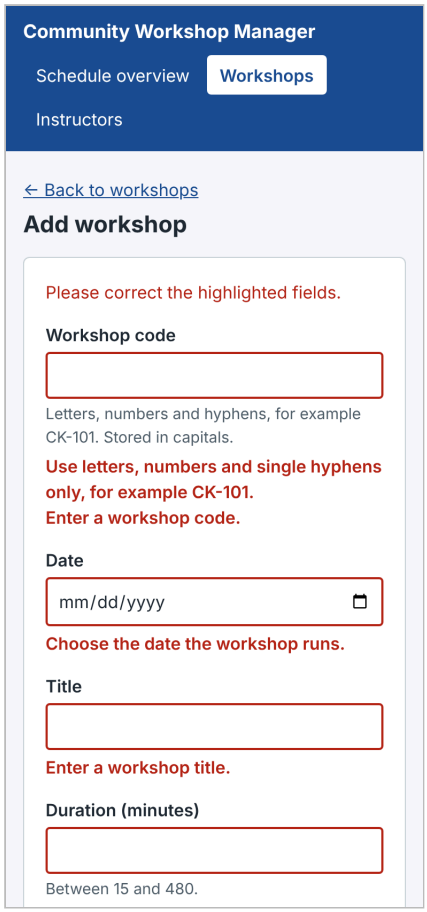


Figure 19. Workshop form with errors at 390 px width. Fields stack in one column.

10. Challenges and how they were solved

JSP compile error that unit tests missed. At first every page set its title with `<c:set>` and then included a header file that declared the JSTL taglibs. All controller tests passed, because MockMvc only checks the view name and never compiles the JSP. The full-stack rendering test failed straight away with a Jasper error saying the prefix `c` had been used before its taglib directive. I moved the directives into `common/taglibs.jspf`, which each page includes on its first line. This showed why a test against the real container was worth adding.

Recognising which constraint failed. The translator first searched only the exception messages for the constraint name. This worked with real H2 errors, but a controller test that built a Hibernate `ConstraintViolationException` directly failed, because there the name is stored in a separate field. The translator now also reads `getConstraintName()` while walking the cause chain.

Error pages and session IDs. The test client for unknown URLs received Spring Boot's JSON error body, since it sent no `Accept` header. With `Accept: text/html`, as a browser sends, the custom `error.jsp` is used (the default whitelabel page is turned off). While testing with curl I also saw `;jsessionid=` added to links, because the flash messages create a session. Setting `server.servlet.session.tracking-modes=cookie` removed it.

Spring Boot 4 test changes. In Spring Boot 4 the test slices moved into separate modules. I added `spring-boot-starter-data-jpa-test` and `spring-boot-starter-webmvc-test`, imported `@DataJpaTest` and `@WebMvcTest` from their new packages, and used `@MockitoBean` because `@MockBean` is no longer available.

Design decision: open session in view off. Leaving Open Session in View on would let the JSP load the workshop's instructor lazily, but it hides extra queries in the view. I turned it off and used an entity graph instead. The trade-off is that any new page that needs related data must ask for it in the repository method.

11. Source code and running the application

Requirements: Java 21. The Maven Wrapper downloads Maven 3.9.11 on first use. From the `community-workshop-manager` folder:

```
./mvnw clean test                                # run the 56 tests
./mvnw clean package                             # build target/community-workshop-manager.war
java -jar target/community-workshop-manager.war # start the application
```

Open `http://localhost:8080/`. Data is kept in `./data`; delete that folder while the application is stopped to start again with the sample data. Tests use a separate in-memory database.